Services, Ingress e Volumes

Exposição de aplicações e armazenamento de dados

Ada · DE-OP-011 Orquestração · Kubernetes | Aula 06

Cronograma das Aulas

01
17/07 · Sexta
Conhecendo o Kubernetes

02
20/07 · Segunda
Mão na Massa: Minikube e kubectl

03
22/07 · Quarta
Pods, Namespaces e Controladores

04
24/07 · Sexta
Primeira Aplicação no K8S

05
27/07 · Segunda
Estratégias de Deploy e Limitação de Recursos

06
29/07 · Quarta
Services, Ingress e Volumes

07
31/07 · Sexta
Componentes e Troubleshooting

08
03/08 · Segunda
Cluster Multi-Master

09
05/08 · Quarta
Concluir atividades do módulo e preencher rubricas

COMBINADO DA TURMA

Por favor!

Bora participar!



Câmera ligada + mão levantada = aula top!

INTERVALO • 16:30 BRT

Combinados da nossa aula

1. Câmeras ligadas.
2. Entrem no horário: início e intervalo.
3. Participem da aula.
4. Tirem dúvidas.
5. Levantem a mão para tirar dúvidas.

TRABALHO FINAL

Entrega Final!

O desafio

- Suba no **minikube** um Pod com uma API e um Banco de Dados à sua escolha.
- A API deve fazer um **CRUD para produtos**.

TABELA produtos

Id	—	INT
Nome	—	VARCHAR(50)

O que significa CRUD

- **C** — **Create** — Criar
- **R** — **Read** — Ler
- **U** — **Update** — Atualizar
- **D** — **Delete** — Excluir

Atualize a versão do Pod de forma **declarativa**, demonstrando:

Rolling Update | **Recreate**

Objetivos de hoje

- Entender por que não acessamos Pods pelo IP direto
- Conhecer os tipos de Service
- Expor uma aplicação na prática
- Compreender o papel do Ingress
- Persistir dados com ConfigMaps e Volumes Persistentes

1. Services

Um endereço estável para seus Pods

Por que o IP do Pod não serve

Quatro razões para nunca confiar no IP direto de um Pod

1

IP privado e efêmero

Cada Pod nasce com um IP interno e temporário, usado só na rede do cluster.

2

Preso à rede interna

Esse IP não é acessível de fora do cluster sem uma camada de roteamento.

3

Muda a cada recriação

Pod que morre ou reescala renasce com um IP novo — a conexão antiga quebra.

4

Um Pod só, sem balanceamento

Apontar para um IP envia todo o tráfego a uma única instância, sem distribuir a carga.

A solução: o Service

Endereço único e estável: um IP fixo e um nome DNS que nunca mudam, mesmo com Pods recriados.

Balanceamento de carga: distribui as requisições entre todos os Pods ativos (via labels e selectors).

Nunca aponte para o Pod — aponte para o Service.

Por que o IP do Pod não serve

```
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl get pods -n aplicacao -o wide
NAME                                READY   STATUS    RESTARTS        AGE   IP            NODE       NOMINATED NODE   READINESS GATES
primeira-aplicacao-b45b9c8b4-4dvpr   1/1     Running   2 (5m16s ago)   16h   10.244.0.46   minikube   <none>           <none>
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05>
```

Fonte da imagem: Ada Tech

Os tipos de Service

O tipo depende de quem precisa acessar — de dentro ou de fora do cluster.

	**ClusterIP**	Padrão. IP virtual estável para a comunicação entre microsserviços (ex.: API ↔ banco).	Interno
	**NodePort**	Abre uma porta (30000–32767) em todos os Nodes. Ideal para dev/testes e exposição rápida.	Externo (Node)
	**LoadBalancer**	Balanceador real da cloud (AWS/GCP/Azure) com IP público. Para produção na internet.	Internet
	**ExternalName**	CNAME para um DNS externo (ex.: banco gerenciado), sem fixar a URL no código.	Saída externa
	**Headless**	Sem IP virtual: o DNS devolve os IPs de cada Pod. Para StatefulSets (Mongo, Cassandra, Kafka).	Interno (por Pod)

Expondo uma aplicação com NodePort

MÃO NA MASSA

Se o namespace não existir crie:

```
$ kubectl create ns aplicacao
```

Suba um Pod nginx para ter uma interface web:

```
$ kubectl run nginx --image nginx:1.14.2 -n aplicacao
```

Exponha o Pod via NodePort:

```
$ kubectl expose pod/nginx -n aplicacao --type=NodePort --port=80
```

Liste os Services:

```
$ kubectl get svc -n aplicacao
```

Veja os services no Minikube:

```
$ minikube service list
```

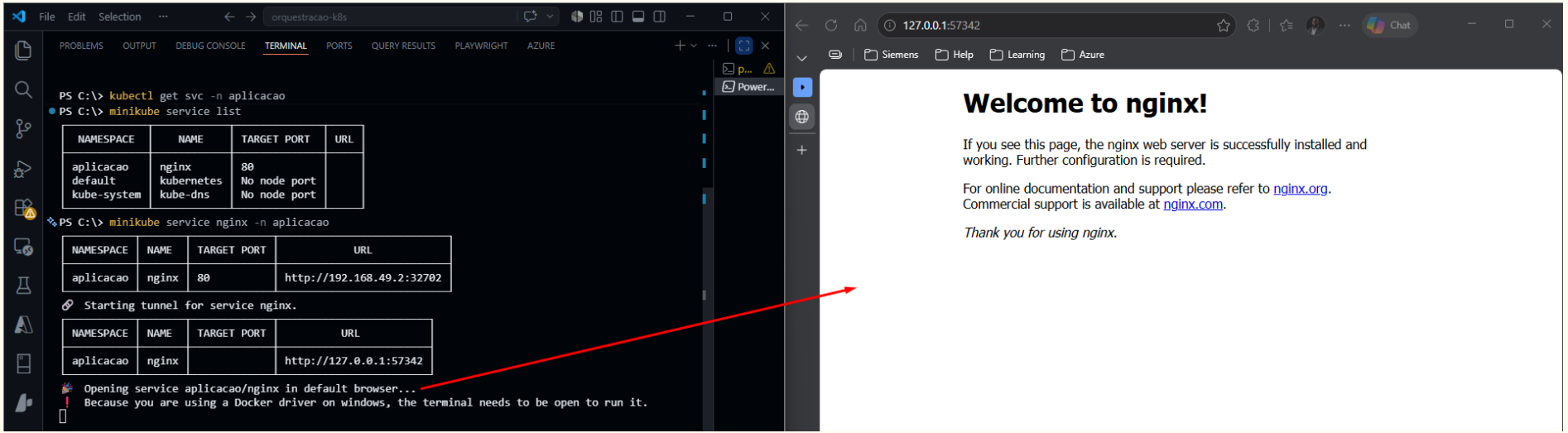
Descubra a URL de acesso e abra o browser pelo Minikube:

```
$ minikube service nginx -n aplicacao
```

Descubra a URL de acesso pelo Minikube:

```
$ minikube service nginx -n aplicação --url
```

Expondo uma aplicação com NodePort (cont.)



Fonte da imagem: Ada Tech

Service como manifesto

nginx-svc.yaml — equivalente declarativo do 'expose'

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx
  namespace: aplicacao
  labels:
    run: nginx
spec:
  type: NodePort
  ports:
  - nodePort: 31756   # porta exposta no node
    port: 80          # porta do Service
    protocol: TCP
    targetPort: 80    # porta do container
  selector:
    run: nginx        # seleciona os Pods com esta label
```

Aplicando o Service de forma declarativa

MÃO NA MASSA

Aplique o manifesto **`nginx-svc.yaml`** declarativamente:

```
$ kubectl apply -f nginx-svc.yaml
```

Confirme que o Service foi criado:

```
$ kubectl get svc -n aplicacao
```

Inspecione os detalhes do Service:

```
$ kubectl describe svc nginx -n aplicacao
```

Liste os Services no Minikube:

```
$ minikube service list
```

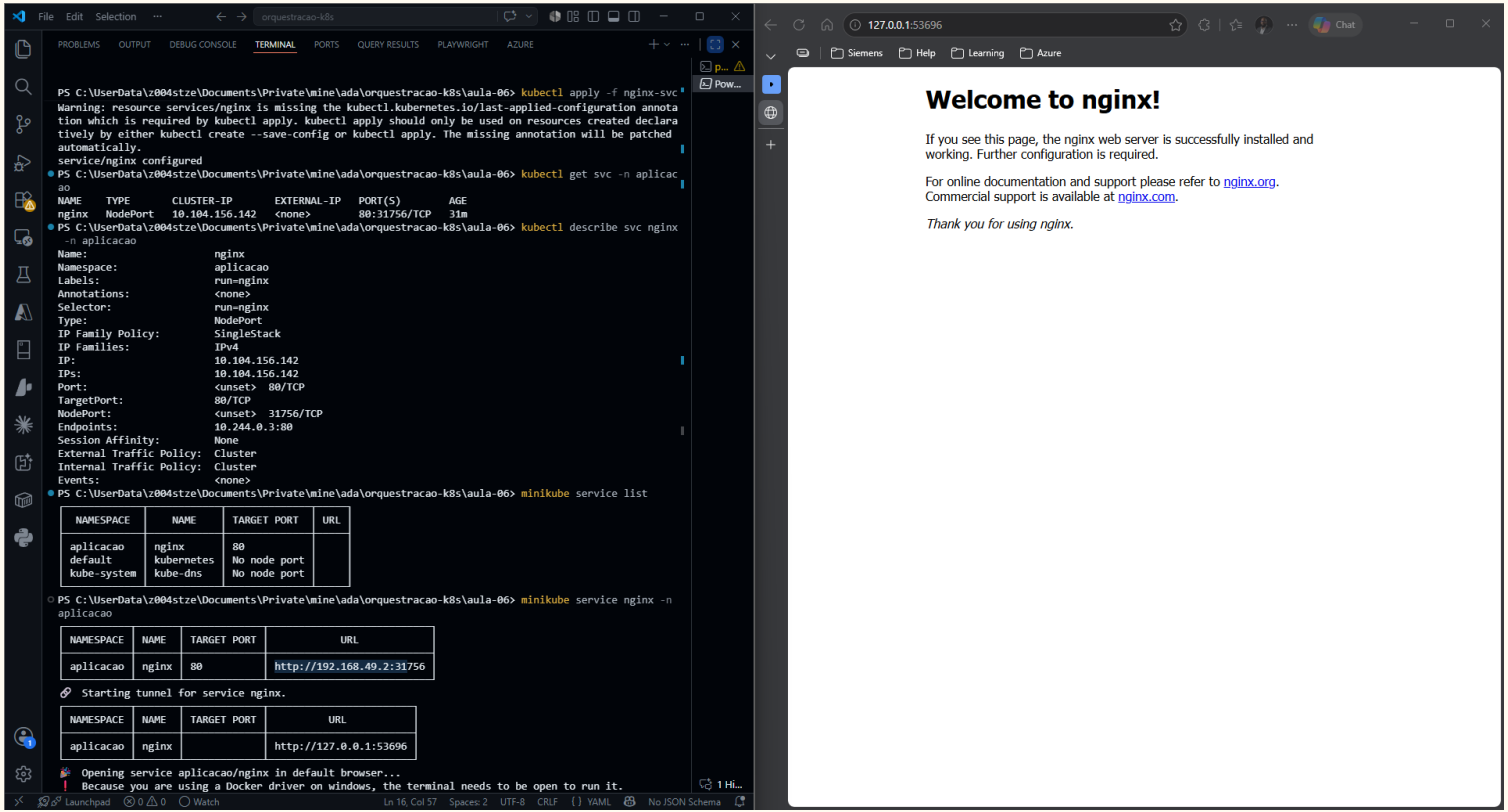
Abra o browser pela URL do Minikube:

```
$ minikube service nginx -n aplicacao
```

Ou obtenha apenas a URL de acesso:

```
$ minikube service nginx -n aplicacao --url
```

Expondo uma aplicação com NodePort (cont.)



Fonte da imagem: Ada Tech

Ingress: um só ponto de entrada HTTP/HTTPS

- **Vários LoadBalancers = caro.** Cada Service `LoadBalancer` gera um balanceador externo cobrado à parte — 10 apps, 10 cobranças.
- **Ingress = UM balanceador.** Um único ponto de entrada na frente de tudo; ele decide internamente o destino de cada requisição.
- **Roteia por host e path.** `app1.com` → Service A; `site.com/api` e `/blog` → Services diferentes.
- **Precisa de um Ingress Controller.** O manifesto é só a regra; quem executa é o `nginx-ingress`, `Traefik` ou `HAProxy`. Sem ele, não tem efeito.
- **Ideal para várias apps web.** Expõe tudo com um único IP externo, usando domínios ou paths distintos.



2. Armazenamento

ConfigMaps e Volumes Persistentes

O problema da efemeridade

- Dados escritos dentro do container somem quando ele morre
- Volumes resolvem: armazenamento anexado ao Pod
- Volumes efêmeros: vivem junto com o Pod
- Volumes persistentes: sobrevivem à morte do Pod
- Focaremos em ConfigMap e Volumes Persistentes

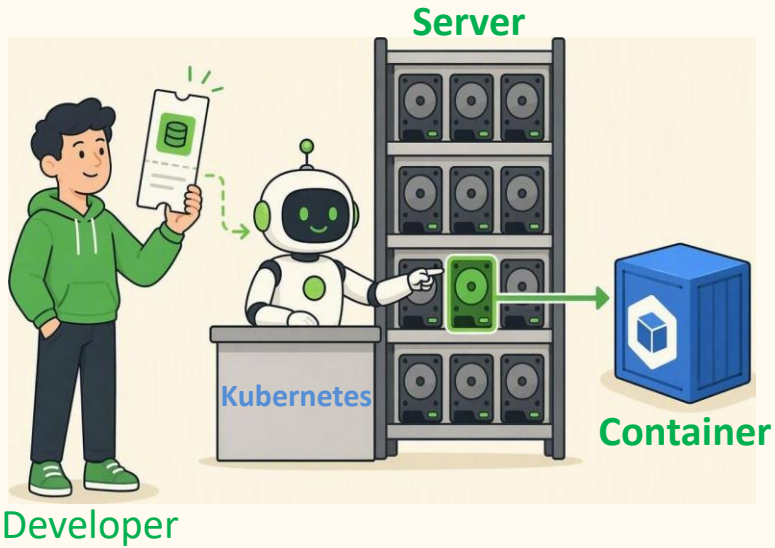
ConfigMap: configuração externa

injeta configuração (não-sensível) nos containers

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  player_initial_lives: "3"
  ui_properties_file_name: "user-interface.properties"
  game.properties: |
    enemy.types=aliens,monsters
    player.maximum-lives=5
  user-interface.properties: |
    color.good=purple
    allow.textmode=true
```

Volumes Persistentes: PV e PVC

- **PV — PersistentVolume.** O “pedaço de disco” real no cluster (ou na nuvem). Recurso de infraestrutura com vida própria, independente de qualquer Pod.
- **PVC — PersistentVolumeClaim.** O pedido do dev: “preciso de 3Gi”. Ele só declara a necessidade; o K8s encontra ou cria um PV que atenda.
- **Estático ou dinâmico.** Estático: o admin deixa PVs “em estoque”. Dinâmico: uma `StorageClass` provisiona o PV sob demanda — o mais comum na cloud.
- **Binding: vínculo 1-para-1.** O K8s liga PVC e PV compatíveis de forma exclusiva; um PV já vinculado não serve a outro PVC.
- **O Pod referencia o PVC.** Nunca o PV direto. Essa abstração esconde se o storage é disco local, EBS, Azure Disk...



Criando PV, PVC e associando a um Pod

MÃO NA MASSA

Crie o PersistentVolume (10Gi):

```
$ kubectl apply -f pv.yaml
$ kubectl get pv task-pv-volume
```

Status Available = ainda não vinculado

Crie o PersistentVolumeClaim (pede 3Gi):

```
$ kubectl apply -f pvc.yaml
$ kubectl get pvc task-pv-claim
```

Agora o PV aparece como Bound. Crie o Pod que usa o PVC:

```
$ kubectl apply -f pod-pvc.yaml
$ kubectl describe pods task-pv-pod
```

Criando PV, PVC e associando a um Pod

```
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-06> kubectl apply -f pv.yaml
persistentvolume/task-pv-volume created
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-06> kubectl get pv task-pv-volume
NAME             CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS      CLAIM   STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
task-pv-volume   10Gi       RWO            Retain           Available           manual         <unset>                          18s
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-06> kubectl apply -f pvc.yaml
persistentvolumeclaim/task-pv-claim created
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-06> kubectl get pvc task-pv-claim
NAME            STATUS   VOLUME           CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
task-pv-claim   Bound    task-pv-volume   10Gi       RWO            manual         <unset>                 14s
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-06> kubectl apply -f pod-pvc.yaml
pod/task-pv-pod created
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-06> kubectl describe pods task-pv-pod
Name:             task-pv-pod
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Tue, 28 Jul 2026 21:05:45 +0100
Labels:           <none>
Annotations:      <none>
Status:           Pending
IP:
IPs:              <none>
Containers:
  task-pv-container:
    Container ID:
    Image:          nginx
    Image ID:
    Port:           80/TCP (http-server)
    Host Port:      0/TCP (http-server)
    State:          Waiting
      Reason:       ContainerCreating
    Ready:          False
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /usr/share/nginx/html from task-pv-storage (rw)
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-689x2 (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   False
  Initialized                 True
  Ready                       False
  ContainersReady             False
  PodScheduled                True
Volumes:
  task-pv-storage:
    Type:       PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)
    ClaimName:  task-pv-claim
    ReadOnly:   false
  kube-api-access-689x2:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
```

Fonte da imagem: Ada Tech

Recapitulando a Aula 06

- ✓ Service dá endereço estável e balanceamento aos Pods
- ✓ Tipos: ClusterIP, NodePort, LoadBalancer, ExternalName, Headless
- ✓ Ingress centraliza o acesso HTTP/HTTPS do cluster
- ✓ ConfigMap externaliza configuração; PV/PVC persistem dados
- ✓ Próxima aula: componentes finais e Troubleshooting

